



INSTITUTO FEDERAL
Rio Grande do Sul

Banco de Dados II

Estruturas de arquivo, indexação e hashing

Prof. Dr. Edimar Manica

edimar.manica@ibiruba.ifrs.edu.br



Roteiro

- Como um SGBD relacional armazena os dados?



Referência

- Referência
 - Título: Sistemas de Banco de Dados
 - Autores: Elmasri & Navathe
 - Edição: 6ª
 - Capítulo: Estruturas de arquivo, indexação e hashing
 - Páginas: 388-423

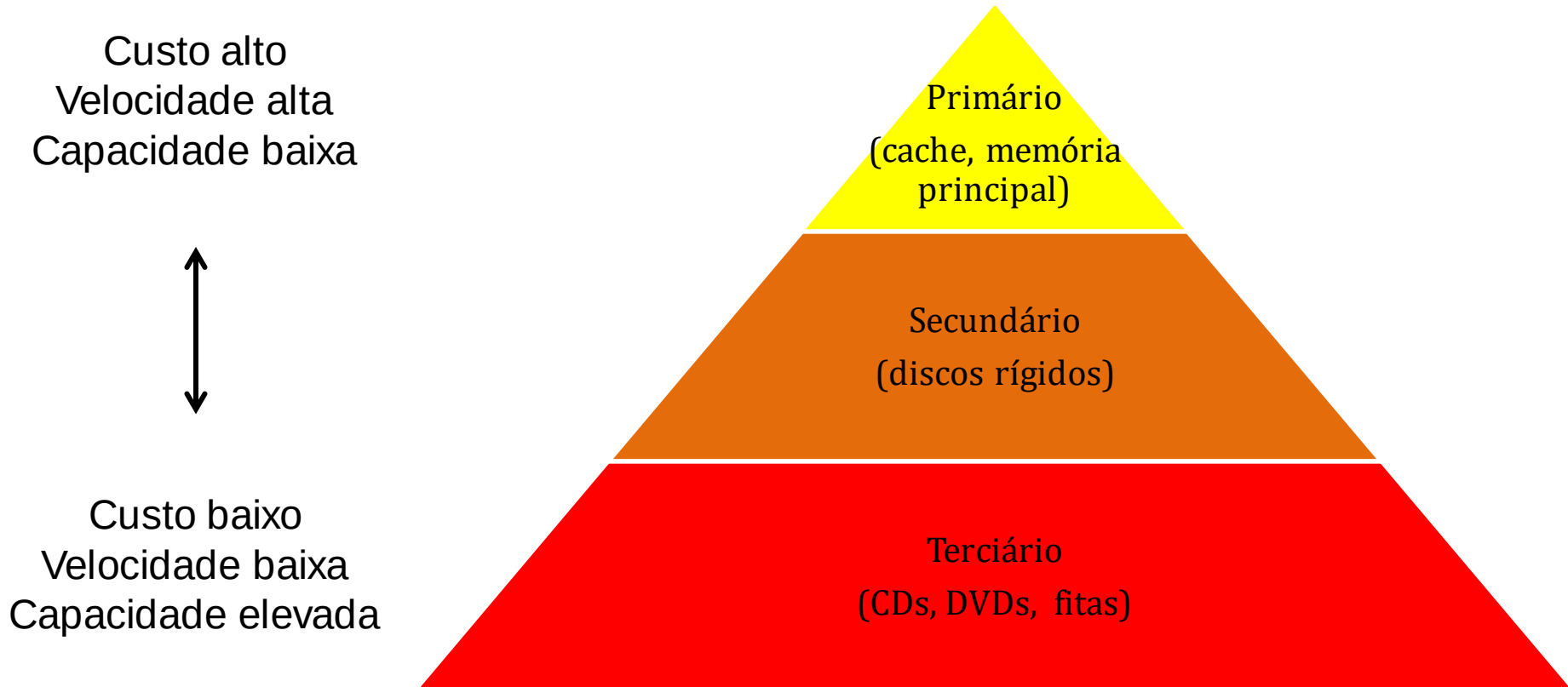
Introdução

- A coleção de dados que compõem um banco de dados computadorizado deve ser armazenada em algum **meio de armazenamento**
- O SGBD pode realizar operações sobre esses dados
 - Recuperar
 - Atualizar
 - Processar

Introdução

- A mídia de armazenamento de computador forma uma hierarquia de armazenamento que inclui duas categorias principais:
 - Armazenamento primário
 - Mídia de armazenamento que pode ser operada diretamente pela unidade central de processamento (CPU)
 - **Vantagem:** fornece acesso rápido aos dados
 - **Desvantagem:** capacidade de armazenamento limitada, mais caros
 - Ex: memória principal e memórias caches menores
 - Armazenamento secundário e terciário
 - **Vantagem:** capacidade de armazenamento maior, menor custo
 - **Desvantagem:** acesso mais lento, os dados não são processados diretamente pela CPU, primeiro, eles precisam ser copiados para o armazenamento primário
 - Ex: discos magnéticos, discos ópticos (CD-ROMs, DVDs) e fitas

Hierarquia de memória e dispositivos de armazenamento



Armazenamento primário

- Memória cache
 - É uma RAM (*Random Access Memory*) estática
 - Mais cara
 - Normalmente utilizada pela CPU para agilizar a execução de instruções de programa
- DRAM
 - *Dinamic* RAM
 - Popularmente chamada de memória principal
 - CPU utiliza para manter instruções de programa e dados
 - + Custo menor que a cache
 - - Volatilidade (perde o conteúdo caso falte energia)
 - - Menor velocidade que a RAM estática

Armazenamento Secundário e Terciário

- Discos magnéticos
 - Discos rígidos
- Discos ópticos
 - Armazenam dados opticamente
 - São lidos por um laser
 - Ex: CD-ROM, DVD
- Fitas
 - Mais baratos da hierarquia
 - Acesso aos dados é muito lento
 - São utilizadas para arquivamento e armazenamento de *backup* dos dados
 - Dados off-line: é necessário carregar uma fita antes que os dados se tornem disponíveis

Capacidade de Armazenamento

- Quilobytes (KB) = 1.000 bytes
- Megabytes (MB) = 1 milhão de bytes
- Gigabytes (GB) = 1 bilhão de bytes
- Terabytes (TB) = 1.000 GB
- Petabyte (PB) = 1.000 terabytes
 - Física, astronomia, ciências terrestres, IOT

Armazenamento de banco de dados

- Residem no armazenamento secundário
 - Discos rígidos
- Partes do banco de dados são carregadas para a memória principal
- Exceções: Bancos de dados em memória (NoSQL)

Conceitos

- **Dados persistentes**

- Dados que precisam persistir por longos períodos de tempo

- **Dados transientes**

- Persistem apenas por um tempo limitado durante a execução do programa

Armazenamento de banco de dados

- A maioria dos BDs é armazenada de maneira persistente em armazenamento secundário porque:
 - Em geral, os BDs não cabem inteiramente na memória principal
 - As circunstâncias que causam perda permanente de dados armazenados surgem com menos frequência para o armazenamento secundário do que para o primário
 - O custo de armazenamento é menor que o primário

Importância

- Os projetistas e o DBA precisam conhecer as vantagens e desvantagens de cada técnica de armazenamento quando projetam, implementam e operam um BD em um SGBD específico
- Em geral, o SGBD tem várias opções disponíveis para organizar os dados
- O processo de projeto físico do BD envolve a escolha das técnicas de organização de dados em particular que mais se ajustam a determinados requisitos da aplicação dentre as opções
- Os implementadores de SGBD devem estudar as técnicas de organização de dados de modo que possam implementá-las de modo eficaz e, portanto, oferecer ao DBA e aos usuários do SGBD opções suficientes

Aplicações de BD típicas

- Só precisam de uma pequena parte do BD de cada vez para processamento
- Etapas
 - Localiza a parte dos dados necessária no disco
 - Copia para a memória principal para processamento
 - Reescreve no disco (se os dados foram alterados)
- Os dados armazenados no disco são organizados como **arquivos de registros**
- Cada registro é uma coleção de valores de dados que podem ser interpretados como fatos sobre entidades, seus atributos e relacionamentos
- Os registros devem ser armazenados em disco de uma maneira que torne possível localizá-los de modo eficiente quando necessário

Organizações de arquivo primário

- Determinam como os registros de arquivo são colocados fisicamente no disco
- Afetam como os registros de arquivo podem ser acessados
- Tipos:
 - Heap
 - Arquivo desordenado
 - Coloca os registros no disco sem qualquer ordem em particular
 - Acrescenta novos registros ao final do arquivo
 - Sequencial
 - Arquivo classificado
 - Mantém os registros ordenados pelo valor de um campo em particular
 - Hashing
 - Usa uma função de hash aplicada em um campo particular para determinar a posição de um registro no disco
 - B-Tree
 - Usam estruturas em árvores

Organizações de Arquivo Secundária

- Estrutura de acesso auxiliar
- Permite acesso eficiente aos registros do arquivo com base em campos alternativos, além dos que foram usados para a organização do arquivo primário
- Veremos na próxima aula
- !Não confundir organização de arquivo primária (forma como os registros são armazenados) com dispositivo de armazenamento primário (cache, memória principal)

Registros

- Os dados costumam ser armazenados na forma de **registros**
- Cada registro contém uma coleção de valores (itens de dados)
- Cada valor é formado por um ou mais bytes e corresponde a um campo em particular do registro
- Os registros normalmente descrevem entidades e seus atributos
- Ex: um registro de funcionário representa a entidade funcionário, e o valor de cada campo no registro especifica algum atributo desse funcionário
- Uma coleção de nomes de campo e seus tipos de dados correspondentes constituem uma definição de **tipo de registro**
- Um tipo de dado, associado a cada campo, especifica os tipos de valores que um campo pode assumir

Registros

- Exemplo de um tipo de registro na linguagem C

```
struct funcionario{  
    char nome[30];  
    char cpf[9];  
    double salario;  
    int cod_cargo;  
    char departamento[20];  
};
```

Registros

- O tipo de dado de um campo normalmente é um dos tipos de dado padrão usados na programação
- O número de bytes exigidos para cada tipo de dado é fixo para determinado sistema de computação
- Ex
 - Inteiro $\rightarrow$ 4 bytes
 - Inteiro longo $\rightarrow$ 8 bytes
 - Número real $\rightarrow$ 4 bytes
 - Booleano $\rightarrow$ 1 byte
 - Data $\rightarrow$ 10 bytes (para formato DD-MM-AAAA)
 - String de tamanho fixo de k caracteres $\rightarrow k$ bytes
 - Strings de tamanho variável $\rightarrow$ número de bytes = número de caracteres no valor do campo + 1

Registros

- E quando os valores de um campo são grandes objetos não estruturados (ex: imagens, vídeos, áudio, etc.)?
 - BLOBs (objetos binários grandes)
 - Um valor BLOB costuma ser armazenado separadamente de seu registro, em um conjunto de blocos de disco, e um ponteiro para o BLOB é incluído no registro

Arquivos

- **Arquivo:** sequência de registros
- **Registros de tamanho fixo:** se cada registro no arquivo tem exatamente o mesmo tamanho (em bytes)
- **Registros de tamanho variável:** se diferentes registros no mesmo arquivo possuem diferentes tamanhos

Registros de tamanho variável

- Motivos:
 - Existência de campos de tamanho variável
 - Ex: NOME_FUNCIONARIO
 - Existência de campos opcionais
 - Ex: RG

Três formatos de armazenamento de registro

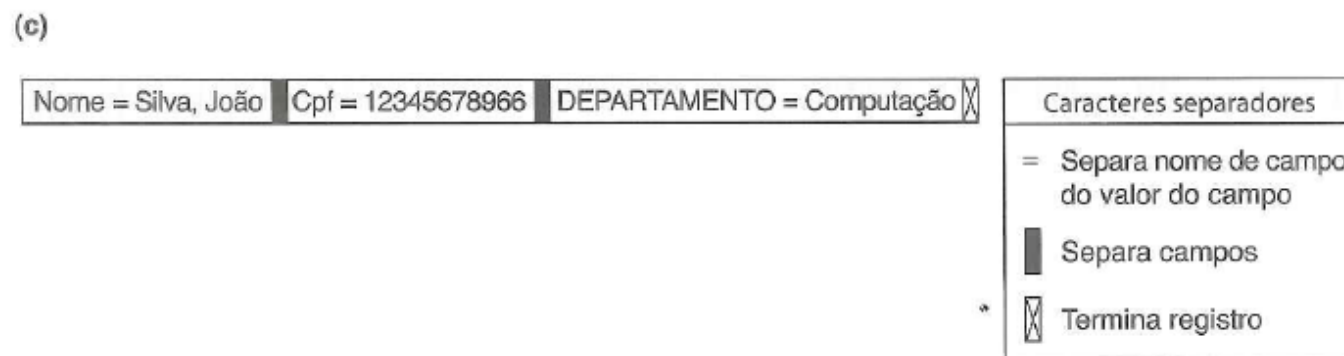
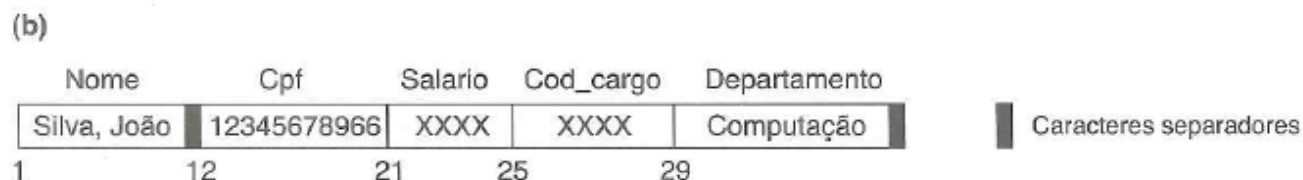
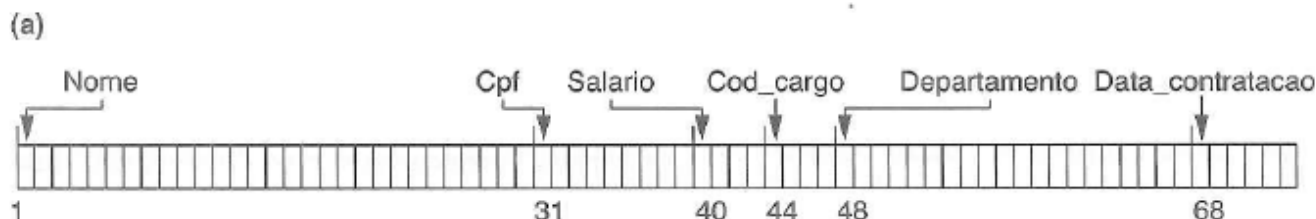


Figura 17.5

Três formatos de armazenamento de registro. (a) Um registro de tamanho fixo com seis campos e tamanho de 71 bytes. (b) Um registro com dois campos de tamanho variável e três campos de tamanho fixo. (c) Um registro de campo variável com três tipos de caracteres separadores.

Blocagem de registros

- Os registros de um arquivo precisam ser alocados a blocos de disco
- Um **bloco** é a unidade de transferência de dados entre o disco e a memória
- Quando o tamanho do bloco é maior que o tamanho do registro, cada bloco terá diversos registros
- Alguns arquivos podem ter registros excepcionalmente grandes que não cabem em um único bloco

Resumo



armazenar



bloco



Pesquisar
Modificar

Exemplo

- **Ação:** alterar o salário da funcionária ANA para 4000
- Armazenamento
 - Registros de tamanho fixo (5 caracteres para nome, 1 caractere para sexo e 4 caracteres para salário, nessa ordem)
 - O tamanho de cada bloco é 20 caracteres (2 registros)
 - & é o indicador de fim do valor do campo nome

Nome					Sexo	Salário			
J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0



Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0



INSTITUTO FEDERAL
Rio Grande do Sul



Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0

Transferindo um
bloco

Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0

Pesquisa linear

Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0

Pesquisa linear

Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

Transferindo
próximo bloco

Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

Pesquisa linear

Exemplo



J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	3	6	0	0
J	E	A	N	&	M	4	0	0	0

A	N	A	&		F	4	0	0	0
J	E	A	N	&	M	4	0	0	0

Modificando na
memória

Exemplo



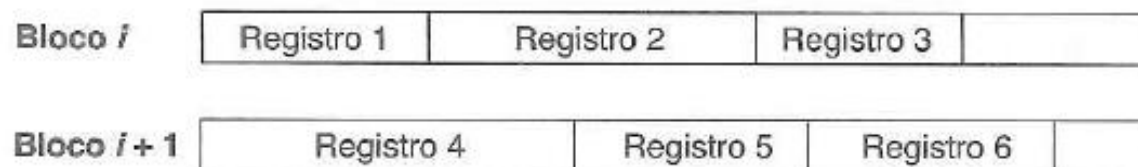
J	O	A	O	&	M	2	0	0	0
L	A	I	S	&	F	0	9	8	0
A	N	A	&		F	4	0	0	0
J	E	A	N	&	M	4	0	0	0

A	N	A	&		F	4	0	0	0
J	E	A	N	&	M	4	0	0	0

Transferindo um
bloco

Tipos de organização de registro

- Não espalhada



- Espalhada



Alocando blocos de arquivo no disco

- Técnicas
 - Alocação contínua
 - Os blocos de arquivo são alocados a blocos de disco consecutivos
 - + Leitura do arquivo inteiro muito rápido usando o *buffering* duplo
 - - Dificulta a expansão do arquivo
 - Alocação ligada
 - Cada bloco do arquivo contém um ponteiro para o próximo bloco de arquivo
 - + facilita a expansão do arquivo
 - - torna a leitura mais lenta
 - Alocação de cluster
 - Combina as duas técnicas anteriores
 - Aloca segmentos (clusters) de blocos de disco consecutivos e os segmentos são ligados
 - Alocação indexada
 - Um ou mais blocos de índice contêm ponteiros para os blocos de arquivos reais

Buffering de blocos

- Quando vários blocos precisam ser transferidos do disco para a memória principal e todos os endereços de bloco são conhecidos, vários *buffers* (áreas) podem ser reservados na memória principal para agilizar a transferência
- Enquanto um *buffer* está sendo lido ou gravado, a CPU pode processar dados em outro *buffer*, pois existe um processador (controlador) de E/S de disco separado que, uma vez iniciado, pode prosseguir para transferir um bloco de dados entre a memória e o disco independente e em paralelo com o processamento da CPU

Cabeçalhos de arquivo

- Contém informações sobre um arquivo, que são exigidas pelos programas do sistema que acessam os registros do arquivo
- Inclui informações para determinar os endereços de disco dos blocos de arquivo
- Para procurar um registro no disco, um ou mais blocos são copiados para os *buffers* da memória principal. Os programas então procuram o registro ou registros desejados nos *buffers*, usando a informação no cabeçalho de arquivo
- Se o endereço do bloco que contém o registro desejado não for conhecido, os programas de pesquisa devem realizar uma pesquisa linear pelos blocos de arquivo. Cada bloco de arquivo é copiado para um *buffer* e pesquisado até que o registro seja localizado ou todos os blocos de arquivo tenham sido pesquisados com sucesso
- **!**O objetivo de uma boa organização de arquivo é localizar o bloco que contém um registro desejado com um número mínimo de transferências de bloco

Operações em arquivos

- **Open**

- prepara o arquivo para leitura e gravação
- Aloca *buffers* apropriados para manter blocos de arquivo do disco
- Recupera o cabeçalho do arquivo
- Define o ponteiro de arquivo para o início do arquivo

- **Reset**

- Define o ponteiro do arquivo aberto para o início do arquivo

- **Find**

- Procura o primeiro registro que satisfaça uma condição de pesquisa
- Transfere o bloco que contém esse registro para o *buffer* da memória principal (se ainda não estiver lá)
- O ponteiro de arquivo aponta para o registro no *buffer* e esse se torna o registro atual

Operações em arquivos

- **Read**

- Copia o registro atual do *buffer* para uma variável no programa do usuário
- Pode avançar o ponteiro do registro atual para o próximo registro no arquivo, que pode precisar ler o próximo bloco de arquivo do disco

- **FindNext**

- Procura o próximo registro no arquivo que satisfaz a condição de pesquisa
- Transfere o bloco que contém esse registro para um *buffer* da memória principal (se ainda não estiver lá)
- O registro está localizado e torna-se o registro atual

Operações em arquivos

- **Delete**
 - Exclui o registro atual
 - Atualiza o arquivo no disco para refletir a exclusão
- **Modify**
 - Modifica alguns valores de campo para o registro atual
 - Atualiza o arquivo no disco para refletir a modificação
- **Insert**
 - Insere um novo registro no arquivo ao localizar o bloco onde o registro deve ser inserido, transferindo esse bloco para um *buffer* da memória principal (se ainda não estiver lá), gravando o registro no *buffer* e gravando o *buffer* em disco para refletir a inserção
- **Close**
 - Completa o acesso ao arquivo liberando os *buffers* e realizando quaisquer outras operações de limpeza necessárias

Operações em arquivos

- **FindAll**
 - Localiza todos os registros no arquivo que satisfazem uma condição de pesquisa
- **Find n**
 - Procura o primeiro registro que satisfaz uma condição de pesquisa e depois continua a localizar os próximos $n-1$ registros que satisfazem a mesma condição
 - Transfere os blocos que contém os n registros para o *buffer* da memória principal (se ainda não estiver lá)
- **FindOrdered**
 - Recupera todos os registros no arquivo em alguma ordem especificada
- **Reorganize**
 - Inicia o processo de reorganização
 - Ex: reordenar

Extra: palavra de máquina

- Processadores de **32 bits** têm a capacidade de processar “**palavras**” (sequência de **bits**) de até **32 bits**
- Processadores de **64 bits** têm a capacidade de processar “**palavras**” (sequência de **bits**) de até **64 bits**
- **Palavra** é um dado natural de uma arquitetura (processador)

	Menor dado	Primeiro agrupamento	Segundo agrupamento
Idioma	Letra	Sílaba	Palavra
Computador	Bit	Byte	Palavra



Extra: palavra de máquina

- Nos idiomas, as palavras variam de tamanho, enquanto que atualmente todas as arquiteturas de computadores possuem palavras com o mesmo número de sílabas (bytes) e como as sílabas também são fixas, temos o mesmo número de letras (bits)
- Quando falamos em palavra estamos falando de um dado que possui um tamanho de bits fixo que aquela arquitetura trabalha melhor
- Em geral estamos falando do tamanho do registrador do processador. Pelo menos dos registradores principais. Podem existir outros secundários para atividades específicas, tais como cálculo de ponto flutuante, vetores, criptografia, etc.

